L1.6 — nginx în fața aplicațiilor

Docker, deploy pe AWS și nginx · pasul 6 din 6

Obții cele trei API-uri pe un singur port public, **80**, prin nginx (`/users/`, `/shop/`, `/school/`); aplicațiile ascultă doar pe `127.0.0.1` al serverului, MySQL nu mai are port, iar `8111`, `8104`, `8105` nu mai răspund din afară. Lucrezi în terminalul din VS Code (zsh), în rădăcina repo-ului (lângă `docker-compose.yaml`). *IP-UL-SERVERULUI* = adresa nouă de la punctul 1.

Browser	Pe server: nginx :80	Pe server, doar local	Container
`http://IP/users/…`	`location /users/`	`127.0.0.1:8111`	`one-user-management-api` :8080, prefix `/users`
`http://IP/shop/…`	`location /shop/`	`127.0.0.1:8104`	`online-shop-api` :8082, prefix `/shop`
`http://IP/school/…`	`location /school/`	`127.0.0.1:8105`	`online-school-api` :8081, prefix `/school`
—	—	—	`mysql` :3306, doar în rețeaua Docker

1. Pornești serverul

În consola EC2: bifezi `devops-initiere` → **Instance state** → **Start instance**.

Trebuie să vezi: `Running`, `3/3 checks passed` și alt **Public IPv4 address** decât la L1.5: îl copiezi în `notite-L1.txt`.

2. Swagger fără adresă scrisă în cod (User Management)

În `OpenApiConfig.java` (`proiecte/one-user-management-java/src/main/java/ro/mycode/user_management/config/`) adresa din Swagger e fixată pe `/`: sub `/users/`, **Try it out** ar trimite la `http://IP/api/users`, fără prefix. O ștergi și Spring o calculează singur din cerere: ștergi rândurile `import io.swagger.v3.oas.models.servers.Server;` și `import java.util.List;`, apoi înlocuiești ultimele două rânduri din `return`

```
        .license(new License().name("Material de curs")))
    .servers(List.of(new Server().url("/").description("Serverul curent")));
```

cu unul singur și salvezi:

```
        .license(new License().name("Material de curs")));
```

```
cd proiecte/one-user-management-java
docker build -t devops-one-user-management-java:1.1 .
docker tag devops-one-user-management-java:1.1 CONTUL-TAU/devops-one-user-management-java:1.1
docker push CONTUL-TAU/devops-one-user-management-java:1.1
cd ../..
```

Trebuie să vezi: `1.1: digest: sha256:…` la push.

3. docker-compose.yaml: porturi doar pe 127.0.0.1, MySQL închis, prefixe

În VS Code înlocuiești tot conținutul lui `docker-compose.yaml` din rădăcină cu textul de mai jos și salvezi (`Cmd+S`):

```
services:
  mysql:
    image: mysql:8.4
    environment:
      MYSQL_ROOT_PASSWORD: root
    volumes:
      - mysql_data:/var/lib/mysql
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost", "-u", "root", "-proot"]
      interval: 5s
      timeout: 5s
      retries: 12
      start_period: 30s
    restart: unless-stopped
  one-user-management-api:
    image: CONTUL-TAU/devops-one-user-management-java:1.1
    ports:
      - "127.0.0.1:8111:8080"
    environment:
      SPRING_DATASOURCE_URL: "jdbc:mysql://mysql:3306/user_management_db?
createDatabaseIfNotExist=true&allowPublicKeyRetrieval=true&useSSL=false"
      SPRING_DATASOURCE_USERNAME: root
      SPRING_DATASOURCE_PASSWORD: root
      SERVER_SERVLET_CONTEXT_PATH: /users
    depends_on:
      mysql:
        condition: service_healthy
    restart: unless-stopped
  online-shop-api:
    image: CONTUL-TAU/devops-online-shop-api-java:1.0
    ports:
      - "127.0.0.1:8104:8082"
```

```
    environment:
      DB_URL: "jdbc:mysql://mysql:3306/online_shop_db?
useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC&createDatabaseIfNotExist=true"
      DB_USERNAME: root
      DB_PASSWORD: root
      SERVER_SERVLET_CONTEXT_PATH: /shop
    depends_on:
      mysql:
        condition: service_healthy
    restart: unless-stopped
  online-school-api:
    image: CONTUL-TAU/devops-online-school-api-java:1.1
    ports:
      - "127.0.0.1:8105:8081"
    environment:
      DB_URL: "jdbc:mysql://mysql:3306/online_school_db?
useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC&createDatabaseIfNotExist=true"
      DB_USERNAME: root
      DB_PASSWORD: root
      SERVER_SERVLET_CONTEXT_PATH: /school
    depends_on:
      mysql:
        condition: service_healthy
    restart: unless-stopped
volumes:
  mysql_data:
```

`127.0.0.1:8111` = portul e deschis doar pentru serverul însuși. `mysql` fără `ports`: aplicațiile ajung la el prin rețeaua Docker. `SERVER_SERVLET_CONTEXT_PATH` = `server.servlet.context-path`: toate adresele aplicației, inclusiv Swagger-ul, se mută sub prefix. `docker compose config --quiet` → **trebuie să vezi** nimic.

4. Site-ul nginx, în repo

În Explorer, în rădăcina repo-ului (lângă `docker-compose.yaml`) → **New File** → `devops-initiere.conf`. Lipești și salvezi:

```
server {
    listen 80 default_server;
    server_name _;

    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;

    location /users/ {
        proxy_pass http://127.0.0.1:8111;
    }

    location /shop/ {
        proxy_pass http://127.0.0.1:8104;
    }

    location /school/ {
        proxy_pass http://127.0.0.1:8105;
    }

    location / {
        default_type text/plain;
        return 200 "devops-initiere: /users/ /shop/ /school/\n";
    }
}
```

`proxy_pass` fără nimic după port = cererea merge cu prefixul intact (`/users/api/users`), cum îl așteaptă aplicația. `proxy_set_header` = aplicația află adresa reală a browserului; din `Host` își face Swagger-ul URL-ul.

```
scp -i ~/.ssh/devops-initiere.pem docker-compose.yaml devops-initiere.conf ubuntu@IP-UL-
SERVERULUI:~/devops-initiere/
```

Trebuie să vezi: `docker-compose.yaml 100%` și `devops-initiere.conf 100%`.

5. Aplicațiile pe loopback, pe server

```
ssh -i ~/.ssh/devops-initiere.pem ubuntu@IP-UL-SERVERULUI
cd ~/devops-initiere
docker compose up -d
docker compose ps
```

Trebuie să vezi: `Pulled` la `devops-one-user-management-java:1.1`, apoi `Recreated` și `Started`; în PORTS `127.0.0.1:8111->8080/tcp`, `127.0.0.1:8104->8082/tcp`, `127.0.0.1:8105->8081/tcp`, iar la `mysql` doar `3306/tcp, 33060/tcp`, fără `0.0.0.0`. `Connection timed out` la ssh = IP-ul tău de acasă s-a schimbat: în regula `22` alegi din nou `My IP`.

După 30 de secunde:

```
curl -I http://127.0.0.1:8111/users/swagger-ui.html
curl -I http://127.0.0.1:8111/swagger-ui.html
```

Trebuie să vezi: la prima `HTTP/1.1 302` cu `Location: /users/swagger-ui/index.html`; la a doua `HTTP/1.1 404`: fără prefix, adresa nu mai există. `Connection refused` = aplicația încă pornește.

6. nginx pe server

```
sudo apt update
sudo apt install -y nginx
systemctl status nginx --no-pager
curl -s http://127.0.0.1 | grep title
```

Trebuie să vezi: `active (running)` și `<title>Welcome to nginx!</title>`: site-ul `default`, pe care îl înlocuiești acum.

```
sudo cp ~/devops-initiere/devops-initiere.conf /etc/nginx/sites-available/devops-initiere
sudo ln -s /etc/nginx/sites-available/devops-initiere /etc/nginx/sites-enabled/devops-initiere
sudo rm /etc/nginx/sites-enabled/default
sudo nginx -t
```

Trebuie să vezi: `syntax is ok` și `test is successful`. Altfel nginx îți spune fișierul și rândul (`… in /etc/nginx/sites-enabled/devops-initiere:12`).

```
sudo systemctl reload nginx
curl http://127.0.0.1/
curl -I http://127.0.0.1/users/swagger-ui.html
```

Trebuie să vezi: `devops-initiere: /users/ /shop/ /school/`, apoi `HTTP/1.1 302` cu `Server: nginx/1.24.0 (Ubuntu)`: nginx a trimis cererea la aplicație. `502 Bad Gateway` = aplicația nu răspunde încă (punctul 5).

7. Security group: doar 22 și 80

În consola EC2: instanța → tab-ul **Security** → link-ul de la **Security groups** → **Edit inbound rules**: **Delete** la cele trei reguli `Custom TCP` `8111`, `8104`, `8105` → **Add rule**: **Type** `HTTP`, **Source** `My IP` → **Save rules**.

Trebuie să vezi: exact două reguli: `SSH` `22` și `HTTP` `80`, amândouă cu IP-ul tău `/32`.

8. Din browser-ul tău, prin nginx

Deschizi `http://`*`IP-UL-SERVERULUI`*`/users/swagger-ui.html` → `GET /api/users` → **Try it out** → **Execute**.

Trebuie să vezi: titlul `User Management API`, **Request URL** `http://IP/users/api/users`, `Code 200` și `"firstName": "Bogdan"`. Răspunsul `devops-initiere: /users/ …` = rulează încă imaginea `1.0`. Pagina se încarcă la nesfârșit = regula `HTTP` lipsește.

`http://`*`IP-UL-SERVERULUI`*`/shop/swagger-ui.html` → `POST /api/auth/login` → **Try it out**, body:

```
{"email": "customer@shop.com", "password": "customer1234"}
```

Trebuie să vezi: titlul `Online Shop API`, **Request URL** `http://IP/shop/api/auth/login`, `Code 200` și `"accessToken": "eyJ…"`.

`http://`*`IP-UL-SERVERULUI`*`/school/swagger-ui.html` → `POST /auth/login` cu body-ul de la L1.4 (`admin@online-school.ro` / `password`) → **Authorize** cu token-ul → `GET /courses` → **Execute**.

Trebuie să vezi: titlul `Online School Cloud API`, **Request URL** `http://IP/school/courses`, `Code 200` și 6 cursuri, primul `"Algebra Fundamentals"`.

Porturile vechi, din terminalul de pe Mac (nu de pe server):

```
curl -m 5 http://IP-UL-SERVERULUI:8111/users/swagger-ui.html
```

Trebuie să vezi: `curl: (28) Connection timed out after 500… milliseconds`: security group-ul nu mai lasă portul, iar pe server el ascultă oricum doar pe `127.0.0.1`.

9. Oprești serverul

`exit` din ssh, apoi în consola EC2: `devops-initiere` → **Instance state** → **Stop instance** → **Stop**.

Trebuie să vezi: `Stopping`, apoi `Stopped`. La următorul **Start** pornesc singure și containerele, și nginx; doar IP-ul e altul.

Gata când…

Cele trei Swagger-e se deschid pe `http://`*`IP-UL-SERVERULUI`*`/users/`, `/shop/` și `/school/`, cu **Request URL** sub prefix și `Code 200`, `:8111` dă `Connection timed out`, iar instanța e `Stopped`. Lecția 1 e gata. **Terminate** șterge serverul cu tot cu disc și nu mai plătești nimic; îl faci doar când îți spune profesorul.

În `notite-L1.txt` adaugi un rând:

```
L1.6: IP public: ÎNLOCUIEȘTE-CU-IP-UL | Request URL la GET /api/users: ÎNLOCUIEȘTE-CU-URL-UL |
curl pe :8111: ÎNLOCUIEȘTE-CU-eroarea | Instance state la final: ÎNLOCUIEȘTE-CU-starea
```